PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Disciplina: Algoritmos e Estruturas de Dados
Obs: Cópias serão desconsideradas, ou seja, a nota será igual a 0 (zero).

- **A lista deve ser entregue no sistema Verde: http://verde.icei.pucminas.br/**
- **Instruções de como usar o verde: https://youtu.be/-iRwi5Ov4W4**
- **Obs: Em todos os programas, deve-se imprimir seguindo o formato descrito nos exemplos**
- **Não coloque o comando Console.ReadLine() no final do Main.**

Exemplo - Analise, entenda e execute o código a seguir. Esse código servirá como referência para a resolução dos exercícios dessa lista.

```
using System;
class Aluno
{
    private string nome;
    private int matricula;
    private char sexo;

    public Aluno(string nome, int matricula, char sexo)
    {
        this.nome = nome;
        this.matricula = matricula;
        this.sexo = sexo;
    }

    public string Nome
    {
        get { return nome; }
        set { nome = value; }
    }

    public int Matricula
    {
        get { return matricula; }
        set { matricula = value; }
    }

    public char Sexo
    {
        get { return sexo; }
        set { sexo = value; }
    }

    public override String ToString()
    {
        return "[nome: " + nome + ", matricula: " + matricula + ", sexo: " + sexo + "]";
    }
}
```

```
using System;
class Fila
{
    private Aluno[] array;
    private int primeiro;
    private int ultimo;

    //Construtor da classe.
    public Fila(int tamanho)
    {
        array = new Aluno[tamanho + 1];
        primeiro = ultimo = 0;
    }

    //Insere um elemento na ultima posicao da fila.
    public void Inserir(Aluno x)
    {
        if (((ultimo + 1) % array.Length) == primeiro)
        {
            throw new Exception("Erro");
        }
```

```
        array[ultimo] = x;
        ultimo = (ultimo + 1) % array.Length;
    }

    // Remove um elemento da primeira posicao da fila
    public Aluno Remover()
    {
        if (primeiro == ultimo)
        {
            throw new Exception("Erro");
        }

        Aluno resp = array[primeiro];
        primeiro = (primeiro + 1) % array.Length;
        return resp;
    }

    // Mostra os elementos da fila
    public void Mostrar()
    {
        int i = primeiro;
        Console.WriteLine("Fila: ");

        while (i != ultimo)
        {
            Console.WriteLine(array[i]);
            i = (i + 1) % array.Length;
        }
        Console.WriteLine();
    }

    // Pesquisa um elemento na fila
    public bool Pesquisar(int matricula)
    {
        bool achou = false;
        for (int i = primeiro; i != ultimo && !achou; i = ((i + 1) % array.Length))
        {
            if (array[i].Matricula == matricula)
            {
                achou = true;
            }
        }
        return achou;
    }
}
```

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Fila filaEspera = new Fila(60);

        Aluno a1 = new Aluno("Maria", 2023100, 'f');
        Aluno a2 = new Aluno("Jose", 2023101, 'm');
        Aluno a3 = new Aluno("Joao", 2023102, 'm');

        //Usando o método ToString implicitamente
        Console.WriteLine(a1);

        filaEspera.Inserir(a1);
        filaEspera.Mostrar();

        filaEspera.Inserir(a2);
        filaEspera.Mostrar();

        filaEspera.Inserir(a3);
        filaEspera.Mostrar();

        bool result = filaEspera.Pesquisar(2023102);
```

```
        if(result)
        {
            Console.WriteLine("O aluno está na fila de espera.\n");
        }
        else
        {
            Console.WriteLine("O aluno NÃO está na fila de espera.\n");
        }

        Aluno temp = filaEspera.Remover();
        Console.WriteLine("Aluno removido:" + temp.Nome);
        filaEspera.Mostrar();

        temp = filaEspera.Remover();
        Console.WriteLine("Aluno removido:" + temp.Nome);
        filaEspera.Mostrar();
    }
}
```

Implemente as classes Lista, Fila e Pilha (USANDO VETORES) e resolva os exercícios a seguir. NÃO devem ser utilizadas as coleções nativas do C#.

Lista Linear

1) Considere que um corredor profissional precisa de um programa para gerenciar os tempos (em horas - double) das maratonas que ele participou. O programa deve disponibilizar as seguintes opções:

1) Inserir um tempo no início da lista
2) Inserir um tempo no final da lista
3) Inserir um tempo numa posição específica da lista. O usuário deve informar o tempo e depois a posição.
4) Remover o primeiro tempo da lista. O programa deve imprimir o tempo removido.
5) Remover o último tempo da lista. O programa deve imprimir o tempo removido.
6) Remover um tempo de uma posição específica na lista. O usuário deve informar a posição do tempo a ser removido. O programa deve imprimir o tempo removido.
7) **Remover a primeira ocorrência de um tempo específico da lista. O usuário deve informar o tempo a ser removido. Adicione na classe Lista um método para remover um elemento específico: void RemoverItem(double x)**
8) **Pesquisar quantas vezes um determinado tempo consta na lista. O usuário deve informar o tempo a ser pesquisado. Adicione na classe Lista um método que retorne o número de elementos com o valor especificado: int Contar(double x)**
9) Mostrar todos os tempos da lista
10) Encerrar o programa

O programa deverá ler a opção informada pelo usuário e executar a operação selecionada. Em seguida o programa deverá ler e executar a operação selecionada. Esse processo deverá ser repetido até que o usuário digite a opção de encerrar o programa. **Obs: Crie uma lista de tamanho 100.**

Exemplo de funcionamento do programa:

```
Op:
1
2
Op:
1
3
Op:
9
3
2
Op:
2
1
Op:
9
```

```
3
2
1
Op:
3
4
1
Op:
9
3
4
2
1
Op:
8
5
0
Op:
2
3
Op:
8
3
2
Op:
4
3
Op:
5
3
Op:
6
1
2
Op:
7
1
Op:
9
4
Op:
10
```

2) Implemente as seguintes classes:

- Crie uma classe Produto com os seguintes atributos:
    - string nome
    - int quant
    - double preco
- Implemente a classe Lista. Deverá ser implementada uma lista de Produtos (Isto é, na classe Lista deverá ter um atributo Produto[] array)
- Implemente a classe Teste contendo o método Main. O programa deve disponibilizar as seguintes opções:

1. Inserir um produto no final da lista. O usuário deve informar o nome, quantidade e o preço do produto.
2. Remover a primeira ocorrência de um produto específico da lista. O usuário deve informar o nome do produto a ser removido. O programa deve imprimir os dados do produto removido ou imprimir *produto não encontrado*

Adicione na classe Lista um método para remover um elemento específico. O método deverá receber o nome do produto e deverá retornar o objeto Produto, ou null (caso o produto não conste na lista): Produto RemoverItem(string nome)

3. Listar os dados de todos os produtos da lista
4. Pesquisar se um produto já consta na lista.O usuário deve informar o nome do produto a ser pesquisado. O programa deve imprimir: *produto cadastrado* OU *produto não cadastrado*

Adicione na classe Lista um método para pesquisar um elemento específico. O método deverá receber o nome do produto: bool Pesquisar(string nome)

5. Encerrar o programa

O programa deverá ler a opção informada pelo usuário e executar a operação selecionada. Em seguida o programa deverá ler e executar a operação selecionada. Esse processo deverá ser repetido até que o usuário digite a opção de encerrar o programa. **Obs: Crie uma lista de tamanho 100.**

Exemplo de funcionamento do programa:

```
Op:
1
arroz
1
47,95
Op:
1
carne
2
100
Op:
1
milho
5
10
Op:
3
arroz 1 47,95
carne 2 100
milho 5 10
Op:
4
carne
produto cadastrado
Op:
4
ervilha
produto não cadastrado
Op:
2
carne
carne 2 100
Op:
2
carne
produto não encontrado
Op:
5
```

Pilha Linear

3) Na notação tradicional de expressões aritméticas pode-se usar parênteses para eliminar ambiguidade

Exemplo:

A + B * C
A + (B * C)
(A+B) * C

A notação polonesa reversa, dispensa o uso de parênteses. Nessa notação os operadores aparecem após os operandos. Ela é utilizada em vários equipamentos eletrônicos, como calculadores e computadores.

Exemplo:

Notação tradicional: A * B – C/D
Notação polonesa reversa: A B * C D / -

Notação tradicional: A * ((B-C)/D)
Notação polonesa reversa: A B C – D / *

Faça um programa que leia uma expressão matemática no formato da notação polonesa reversa, e imprima o resultado da expressão. Utilize a estrutura de dados pilha. Considere que a expressão poderá ter apenas as operações básicas: soma, subtração, multiplicação e divisão.

Exemplo de funcionamento do programa:

```
3572-*4/+
9,25
```

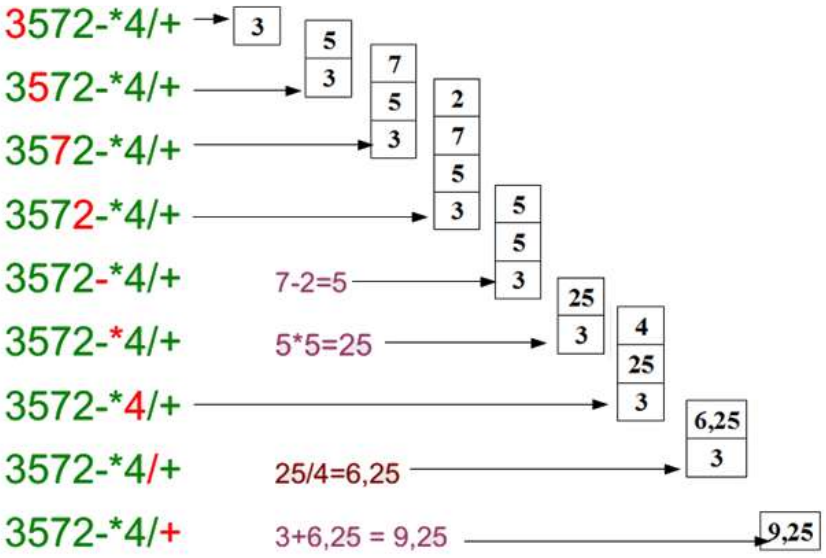
Dicas para desenvolvimento do código:

- **Converter de char para int:**
  int num = (int)Char.GetNumericValue(letra);

- **Acessar posições de uma string:**
  string nome1 = "Maria";
  char letra = nome1[0];

Para avaliar a expressão deve-se seguir esses passos:

- Percorrer a expressão:
  - Se encontrar um operando, empilhar
  - Se encontrar um operador, desempilhar os dois operandos, aplicar a operação e empilhar o resultado
  - Ao final, o resultado estará no topo da pilha



Fila Linear

4) Faça um programa que simule uma fila de impressão. O programa deve disponibilizar as seguintes opções:

1) O usuário deve fornecer o nome e o número de páginas do arquivo que deseja imprimir e este arquivo será inserido na fila de impressão, e ficará lá até que seja executada a sua impressão.
2) O primeiro arquivo da fila de impressão será "encaminhado" para a impressão, ou seja, será retirado da fila (você não precisa imprimir nada, apenas retire o pedido da fila de impressão). O programa deve imprimir o nome do arquivo que será impresso.
3) Deve ser exibido na tela as informações de todos os arquivos que estão na fila de impressão.
4) Encerrar o programa

Para desenvolvimento desse programa, crie as classes Arquivo (atributos nome e número de páginas), Fila e Teste.
Obs: Crie uma fila de tamanho 100.

Exemplo de funcionamento do programa:

```
Op:
1
prova1.pdf
1
Op:
1
prova2.pdf
3
Op:
3
prova1.pdf 1
prova2.pdf 3
Op:
1
lista.pdf
2
Op:
3
prova1.pdf 1
prova2.pdf 3
lista.pdf 2
Op:
2
prova1.pdf
Op:
2
prova2.pdf
Op:
3
lista.pdf 2
Op:
4
```